

<통신시스템 캡스톤 디자인1>

6주차 보고서

얼굴 인식 모델(DeepFace) 연동 및
등록/미등록 인물 판별 구조 구현

-오석준-

체크리스트

DAY	주요 계획	목표	비고
3.31	Raspberry Pi OS 기반 환경 점검 SSH 원격 접속 구성 네트워크 및 IP 확인 Python/pip/venv 환경 구성, YOLO 설치 및 실행 가능성 검토	라즈베리파이 개발 환경 구축 완료 YOLO 실행 오류 (Illegal instruction) 확인	완료
4.7	PC 기반 FastAPI 서버 구축 기본 API(GET/POST) 작성 라즈베리파이↔서버 통신 확인	라즈베리파이와 PC 서버 간 통신 가능 상태 확보	완료
4.14	USB 웹캠(Logitech C920) 연결 카메라 인식 확인 OpenCV 기반 영상 캡처 및 이미지 저장	영상 입력 환경 구축 완료	완료
	라즈베리파이에서 캡처한 이미지를 PC 서버로 전송 HTTP POST 방식 업로드 기능 구현 라이브 스트리밍 기능 구현	이미지 전송 기능 구현	
5.8	PC 서버에서 YOLOv8n/v8s 실행 환경 구성 이미지 기반 객체 탐지 테스트	객체 탐지 기능 1차 구현	완료
5.19	등록 얼굴 데이터 수집, 얼굴 검출 및 특 정 추출 방식 설계, 등록 인물과 미등록 인물을 구분하기 위한 얼굴 인식 구조 설계, 결과 저장을 위한 DB 구조 설계	얼굴 인식 엔진 및 데 이터 저장 체계 확립	완료
	Opencv 기반 얼굴 검출과 특징 비교 기 능 구현, 등록 인물/미등록 인물 판별 테 스트, yolo 사람 탐지 결과와 얼굴 인식 결과를 결합한 판별하는 로직을 구현	등록 인물과 외부인 구 분 기능 구현, 객체 탐 지와 얼굴 인식을 결합 한 판별 구조 확인	
5.26	이벤트 정의 및 알림 기능 구현 침입 발생 시 로그/알림 생성	이벤트 기반 시스템 완성	
6.2	전체 시스템 통합 테스트 오류 수정 및 성능 개선	1학기 1차 완성본 도출	

목 차

1. 6주차 수행 개요
2. 등록 얼굴 데이터 수집 및 정리
 - 2.1 등록 인물 선정
 - 2.2 얼굴 이미지 수집 방식
 - 2.3 등록, 미등록 얼굴 폴더 구조 구성
3. 얼굴 인식 모델 적용 구조
 - 3.1 YOLO 객체 탐지와 얼굴 인식의 역할 구분
 - 3.2 DeepFace 기반 얼굴 인식 방식
 - 3.3 ArcFace 모델 선정 이유
4. YOLO 탐지 결과와 얼굴 인식 결합
 - 4.1 코드 설명
 - 4.2 결과 확인 후 수정
5. DB 저장 구조 설계
 - 5.1 registered_person 테이블 설계
 - 5.2 detection_log 테이블 설계
6. 6주차 결과 및 한계
7. 7주차 계획

1. 6주차 수행 개요

6주차에서는 5주차에 구현한 YOLO 기반 사람 객체 탐지 결과를 바탕으로, 탐지된 사람이 등록 인물인지 미등록 인물인지 구분하기 위한 얼굴 인식 기능을 설계하고 기본 구현 및 테스트를 수행하였다.

이를 위해 등록 인물의 얼굴 이미지를 수집하고, 인물별 폴더 구조를 구성하였다. 또한 DeepFace 기반 ArcFace 모델을 활용하여 입력 얼굴과 등록 얼굴 간의 유사도를 비교하는 방식을 적용하였다.

이후 YOLO의 person 탐지 결과와 얼굴 인식 결과를 결합하는 구조를 설계하고, 탐지 결과와 인물 정보를 저장하기 위한 MySQL 기반 DB 구조를 설계하였다.

2. 등록 얼굴 데이터 수집 및 정리

2.1 등록 인물 선정

초기 얼굴 인식 테스트를 위해 본인을 등록 인물로 설정하였다.
등록 인물은 person01로 지정하였습니다.

2.2 얼굴 이미지 수집 방식

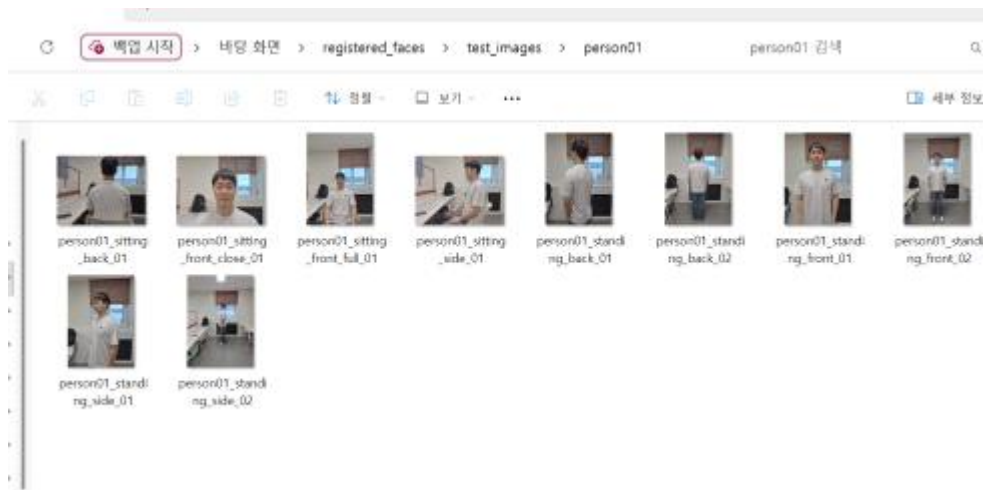
얼굴 이미지는 다양한 인식 조건을 확인하기 위해 정면, 측면, 원거리, 조명 변화가 있는 상황으로 나누어 수집하였다.

이는 얼굴 인식 모델이 한 가지 조건의 이미지에만 의존하지 않고, 다양한 촬영 조건에서도 등록 인물 여부를 비교할 수 있도록 하기 위한 것이다.

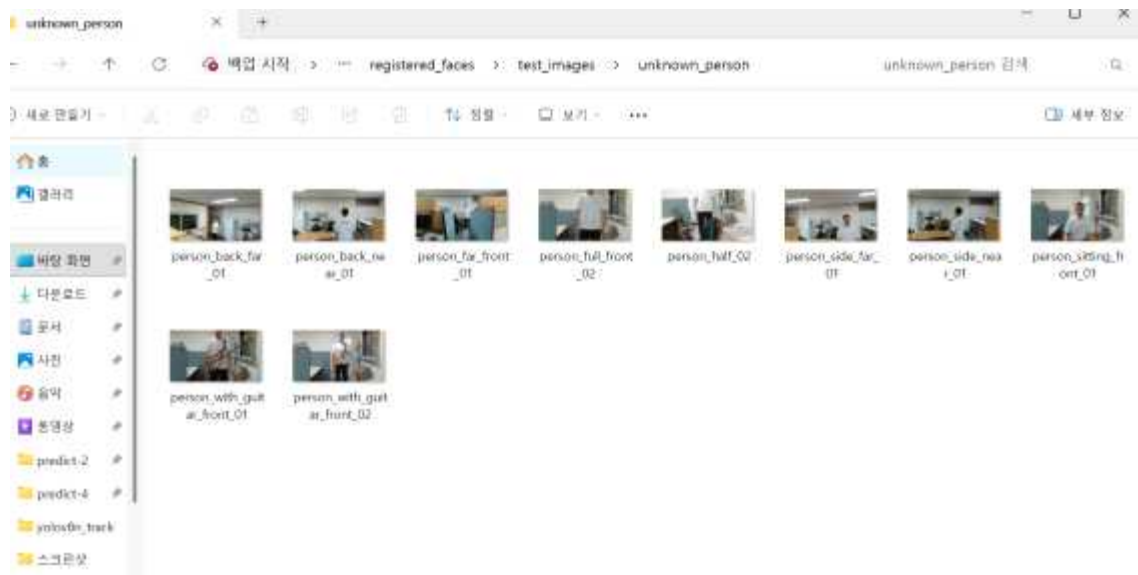
2.3 등록 얼굴 폴더 구조 구성

등록 얼굴 이미지, 미등록 얼굴 이미지는 로컬 폴더 기반으로 관리하였다.

등록 얼굴 이미지



미등록 얼굴 이미지



3. 얼굴 인식 모델 적용 구조

3.1 YOLO 객체 탐지와 얼굴 인식의 역할 구분

YOLO는 이미지 또는 영상에서 사람 객체의 존재 여부를 탐지하는 역할을 수행한다.

그러나 YOLO만으로는 탐지된 사람이 등록 인물인지 미등록 인물인지 판단할 수 없다.

따라서 YOLO로 탐지된 person 영역을 기반으로 얼굴 인식 단계를 추가하여, 탐지된 사람이 등록 인물인지 미등록 인물인지 판별하는 구조로 확장하였다.

3.2 DeepFace 기반 얼굴 인식 방식

DeepFace는 Python 기반 얼굴 인식 라이브러리로, 얼굴 검출, 얼굴 정렬, 얼굴 특징 추출, 얼굴 비교 기능을 제공한다. 사용자는 복잡한 딥러닝 모델을 직접 구현하지 않아도 DeepFace.verify()와 같은 함수를 이용하여 두 얼굴 이미지가 동일 인물인지 비교할 수 있다.

DeepFace는 입력된 얼굴 이미지를 분석하여 얼굴의 특징을 벡터 형태로 추출하고, 등록 얼굴 이미지와 입력 얼굴 이미지 간의 거리를 계산한다. 이때 두 얼굴의 특징 거리가 가까울수록 동일 인물일 가능성이 높고, 거리가 멀수록 다른 인물일 가능성이 높다.

본 프로젝트에서는 YOLO를 통해 사람 객체를 탐지한 뒤, 탐지된 사람이 등록 인물인지 미등록 인물인지 판단하기 위해 DeepFace를 활용하였다. YOLO는 사람의 존재 여부를 탐지하는 역할을 수행하고, DeepFace는 탐지된 사람이 누구인지 확인하는 역할을 수행한다.

구분	역할
YOLO	이미지 또는 영상에서 사람 객체 탐지
DeepFace	입력 얼굴과 등록 얼굴 비교
얼굴 특징 벡터	얼굴의 고유한 특징을 수치화한 값
distance	두 얼굴 특징 벡터 간의 차이
verified	동일 인물 여부 판단 결과

3.3 ArcFace 모델 선정 이유

DeepFace는 여러 얼굴 인식 모델 백엔드를 지원한다. 대표적으로 VGG-Face, Facenet, OpenFace, DeepFace, DeepID, Dlib, ArcFace 등이 있다. 각 모델은 얼굴 특징을 추출하는 방식과 정확도, 처리 속도, 모델 크기에서 차이가 있다.

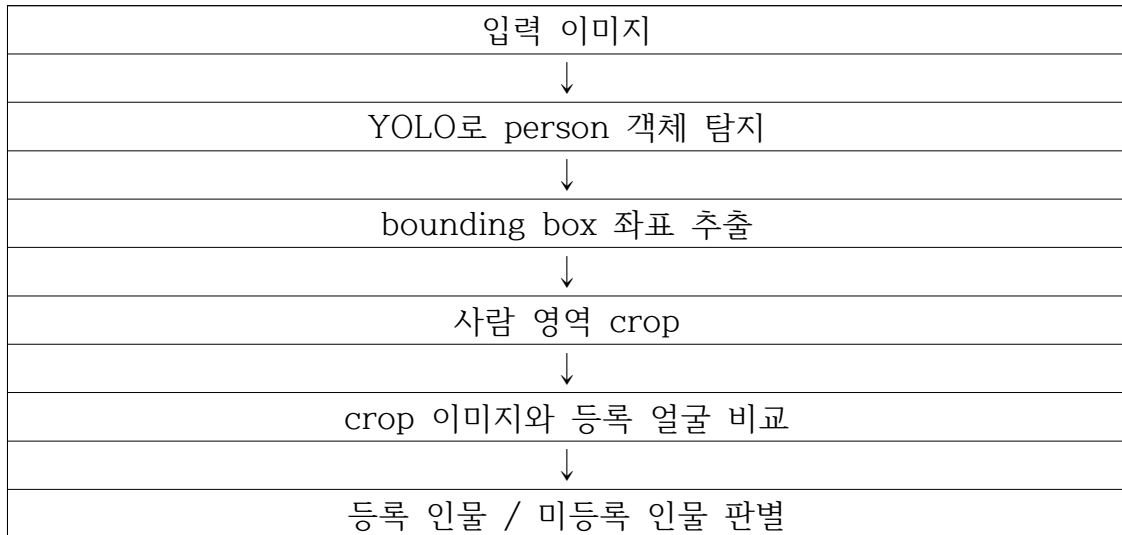
본 프로젝트에서는 이 중 ArcFace를 우선 적용 모델로 선정하였다. ArcFace는 얼굴 이미지를 특징 벡터로 변환한 뒤, 두 얼굴 간의 유사도를 비교하는 장점을 가진 모델이다. 특히 등록 인물과 입력 인물의 얼굴 특징을 비교하여 동일 인물 여부를 판단하는 데 적합하다.

본 프로젝트의 목적은 단순히 얼굴을 검출하는 것이 아니라, YOLO로 탐지된 사람이 등록 인물인지 미등록 인물인지 구분하는 것이다. 따라서 얼굴 특징 비교 성능이 중요하며, 이러한 목적에 맞게 ArcFace를 선정하였다.

출처:[deepface github](https://github.com/deepface/deepface), [arcface 활용](#)

4. YOLO 탐지 결과와 얼굴 인식 결합

전체 흐름



※crop은 원본 이미지에서 필요한 부분만 잘라내는 것

4.1 초기 YOLO + DeepFace 결합 코드 작성

처음에는 5주차에서 작성한 YOLO 사람 탐지 코드를 기반으로, 탐지된 사람 영역을 DeepFace에 입력하는 구조를 작성하였다.

YOLO는 이미지에서 person 객체를 탐지하고, DeepFace는 탐지된 사람이 등록 인물인지 비교하는 역할을 수행하였다.

#YOLO 모델들을 사용하였다.

```
models = {  
    "yolov8n": YOLO("yolov8n.pt"),  
    "yolov8s": YOLO("yolov8s.pt")  
}
```

등록 인물과 미등록 인물 테스트를 모두 수행하기 위해 테스트 이미지
폴더를 person01과 unknown_person으로 구분하였다

```
TEST_DIRS = {  
    "person01": Path("test_images/person01"),  
    "unknown_person": Path("test_images/unknown_person")  
}
```

#DeepFace 공식 threshold 설정에서 ArcFace의 cosine distance 기준값은
0.68이다. 따라서 초기 실험에서는 이를 참고하여 threshold를 0.68로
설정하였다.

```
THRESHOLD = 0.68
```

#YOLO 실행 시 classes=[0] 옵션을 적용하여 사람 객체만 탐지하도록
하였다.

```
results = yolo_model(  
    str(img_path),  
    classes=[0],  
    save=True,  
    project="results",  
    name="yolo_person_face",  
    exist_ok=True  
)
```

```
distance = result["distance"]
```

```
verified = result["verified"]
```

```
-----  
#탐지된 person bounding box 좌표를 이용하여 사람 영역을 crop하였다
```

```
x1, y1, x2, y2 = box.xyxy[0].cpu().numpy().astype(int)
```

```
person_crop = image[y1:y2, x1:x2]
```

```
cv2.imwrite(str(crop_path), person_crop)
```

```
-----  
#crop 이미지를 DeepFace의 ArcFace 모델에 입력하여 등록 얼굴 이미지와  
비교하였다.
```

```
verify_result = DeepFace.verify(  
    img1_path=str(crop_path),  
    img2_path=str(registered_img),  
    model_name="ArcFace",  
    enforce_detection=True  
)
```

```
-----  
#YOLO 탐지 시간을 초 단위가 아니라 밀리초(ms) 단위로 저장하도록 수정
```

```
start_time = time.perf_counter()
```

```
results = yolo_model(  
    str(img_path),  
    classes=[0],  
    save=False,  
    verbose=False  
)
```

```
inference_ms = (time.perf_counter() - start_time) * 1000  
-----
```

#YOLO가 사람 객체를 탐지하더라도 얼굴이 보이지 않는 뒷모습이나 원거리 이미지에서는 DeepFace가 얼굴을 검출하지 못하는 경우가 있었다.

이러한 경우를 미등록 인물로 단정하면 부정확하므로, 본 실험에서는 별도로 Unidentified로 분류하였다.

registered 조건 - verified=True이고 distance가 threshold보다 낮을 때만 등록 인물

```
if best_match is None:
```

```
    final_result = "Unidentified"
```

```
elif best_verified is True and best_distance < THRESHOLD:
```

```
    final_result = "Registered"
```

```
else:
```

```
    final_result = "Unknown"
```

```
-----  
#결과를 직관적으로 확인하기 위해 원본 이미지 위에 YOLO bounding  
box와 DeepFace 판별 결과를 함께 표시하였다.
```

```
cv2.rectangle(annotated, (x1, y1), (x2, y2), color, 2)
```

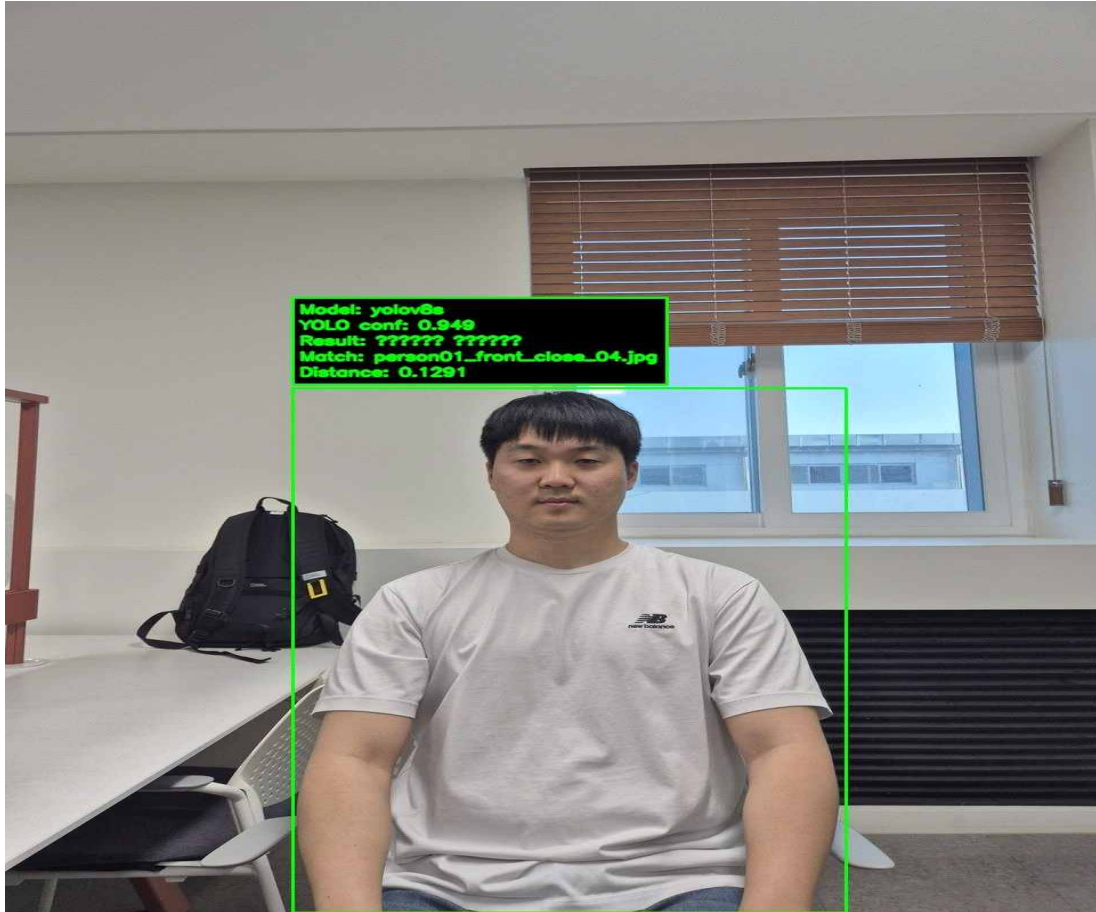
```
text_lines = [  
    f"Model: {model_name}",  
    f"YOLO conf: {yolo_conf:.3f}",  
    f"Time: {inference_ms:.2f} ms",  
    f"Result: {final_result}"  
]
```

```
]  
-----
```

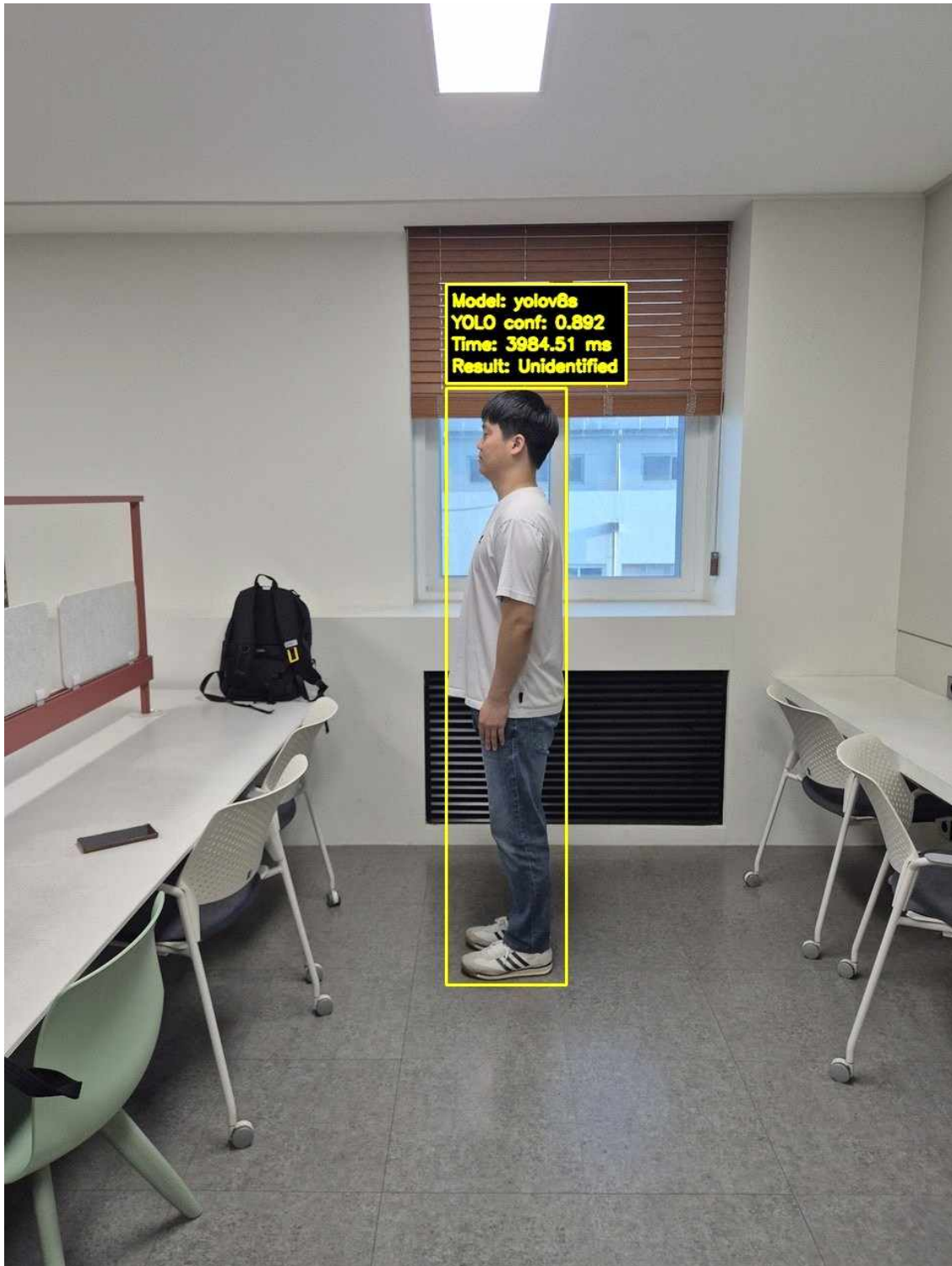
출처:[deepface 활용 사례](#), [YOLO와 인식 모델 활용](#), [얼굴 인식 training](#),
[오픈소스를 통한 deepface 맛보기](#)

결과

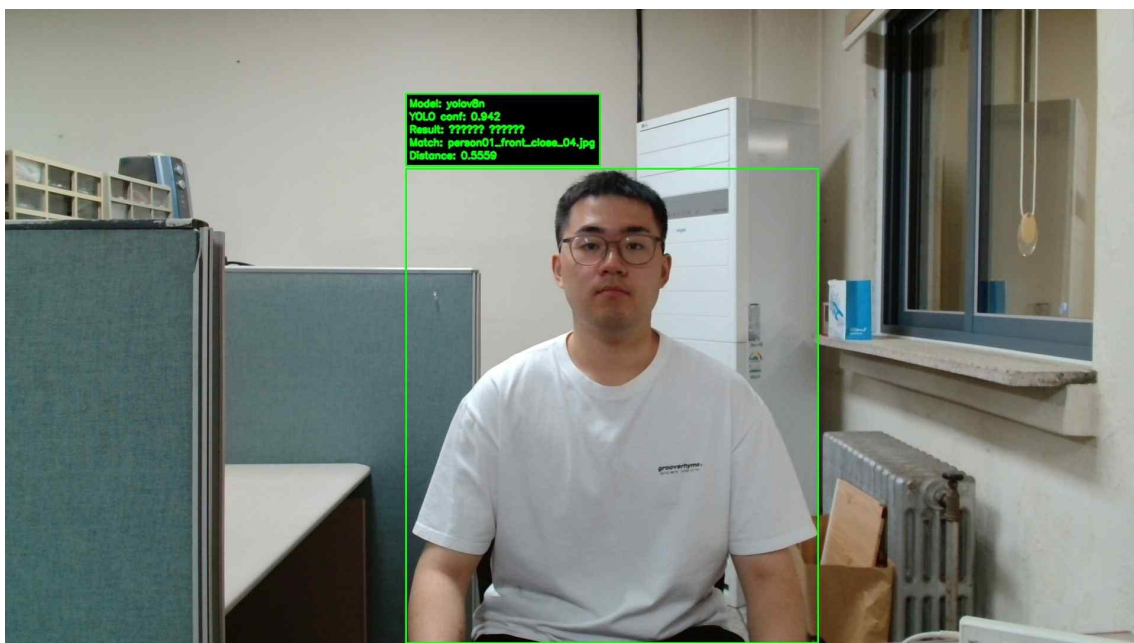
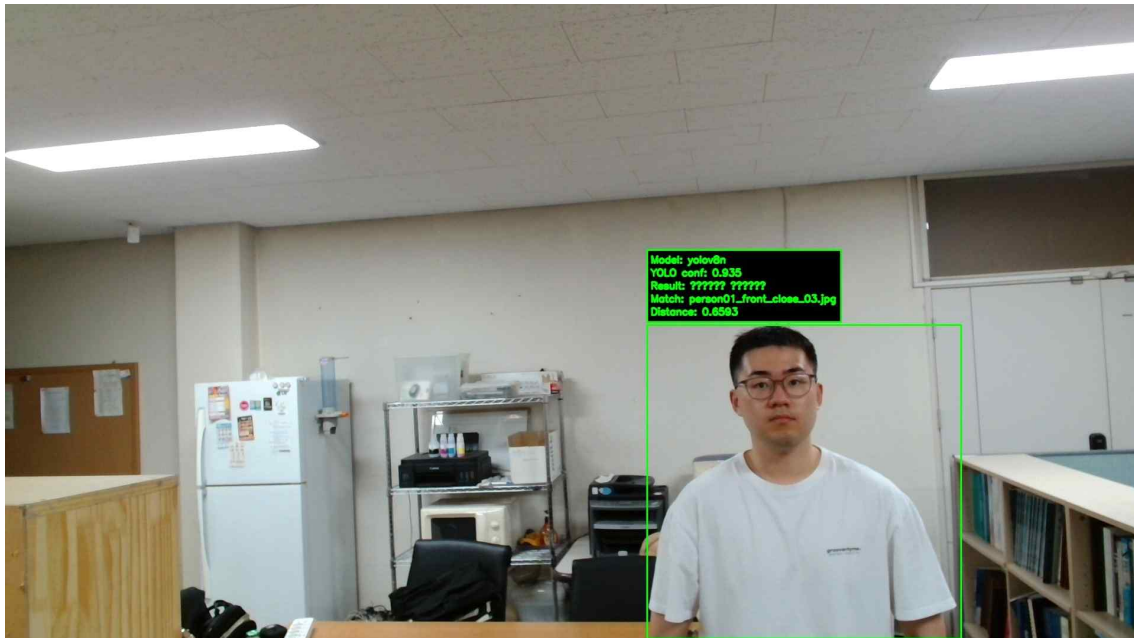
YOLOv8s로 정상적으로 인식된 모습



측면은 인식이 되지 않는 모습



미등록 인물인데도 threshold보다 낮아서 등록인물로 인식된 모습



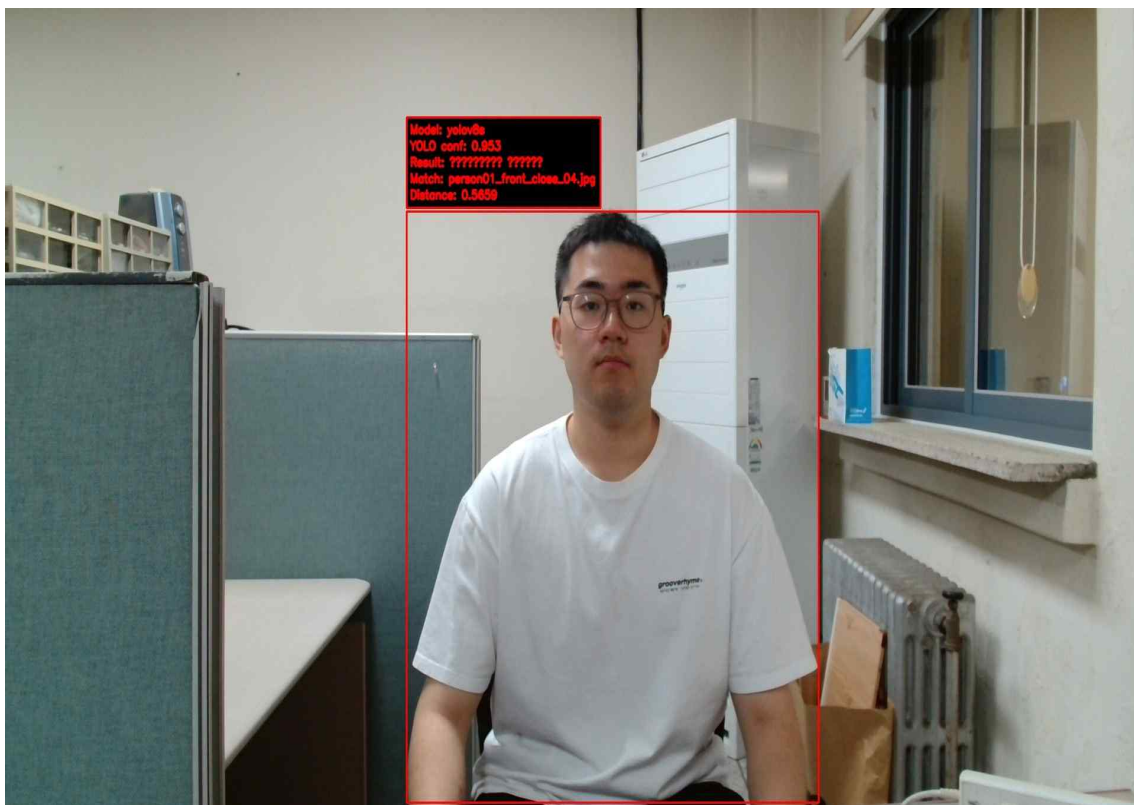
결과 확인 후 수정 (1)

#초기 기준값 0.68을 적용했을 때 일부 미등록 인물이 등록 인물로 판별되는 오탐이 발생하였다.

#이에 따라 등록 인물과 미등록 인물의 distance 분포를 비교하였고, 본 실험에서는 오탐을 줄이기 위해 threshold를 0.50으로 조정하였다.

THRESHOLD = 0.50

수정된 결과




```
#bounding box
x1, y1, x2, y2 = box.xyxy[0].cpu().numpy().astype(int)
person_crop = image[y1:y2, x1:x2]
cv2.imwrite(str(crop_path), person_crop)
```



#초기에는 YOLO가 탐지한 person bounding box 영역을 그대로 crop하여 DeepFace에 입력하였다. 그러나 측면 이미지에서는 얼굴이 bounding box의 가장자리에 위치하거나 일부가 잘려 얼굴 검출이 실패하는 경우가 있었다. 이를 보완하기 위해 bounding box의 가로·세로 길이를 기준으로 약 10%의 여유 공간을 추가하였다.

#또한 얼굴은 일반적으로 사람 bounding box의 상단부에 위치하므로, 여유 공간이 포함된 전체 사람 영역인 full crop과 함께 상단 45% 영역인 upper crop을 추가로 생성하였다.

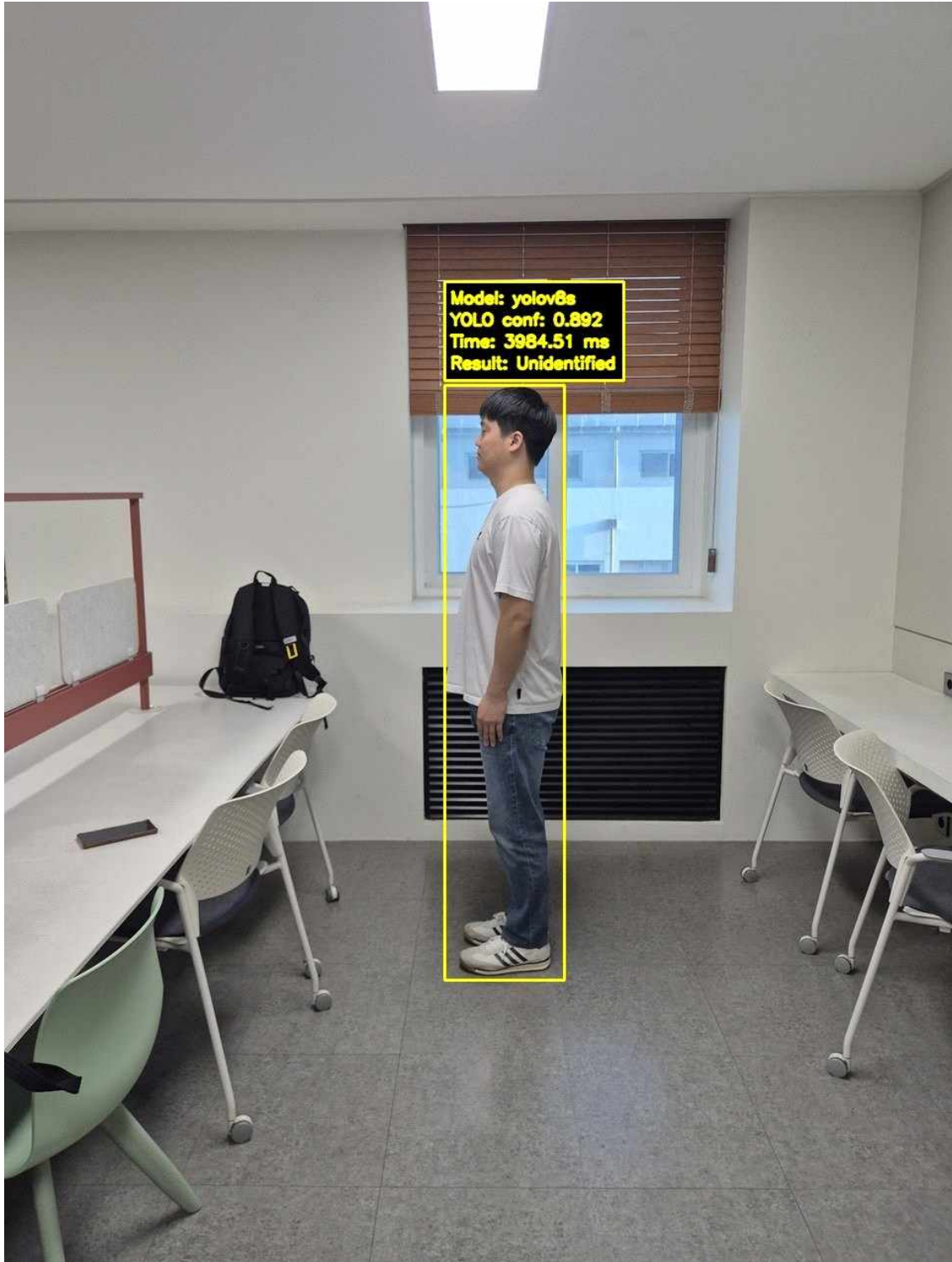
full crop은 사람 전체 영역을 보존하기 위한 것이고, upper crop은 얼굴 검출 가능성을 높이기 위한 보조 입력 이미지이다.

BOX_MARGIN_RATIO=0.1과 upper crop 45%는 얼굴 잘림을 줄이고 불필요한 배경 포함을 최소화하기 위해 설정값이다.

```
BOX_MARGIN_RATIO = 0.1
box_w = x2 - x1
box_h = y2 - y1
pad_x = int(box_w * BOX_MARGIN_RATIO)
pad_y = int(box_h * BOX_MARGIN_RATIO)
```

```
ex1 = max(0, x1 - pad_x)
ey1 = max(0, y1 - pad_y)
ex2 = min(w, x2 + pad_x)
ey2 = min(h, y2 + pad_y)
upper_h = int(crop_h * 0.45)
full_crop = image[ey1:ey2, ex1:ex2]
upper_crop = full_crop[:upper_h, :]
```

하지만 crop 방식 개선 후에도 얼굴 검출 결과는 여전히 인식되지 않았습니다.



결과 확인 후 수정(2)

DeepFace의 detector_backend를 변경하며 결과를 비교하였다.

detector_backend는 ArcFace로 얼굴 특징을 비교하기 전에 crop 이미지 안에서 얼굴 위치를 검출하는 역할을 한다.

본 실험에서는 RetinaFace, OpenCV, MediaPipe 등을 비교하였으며, 현재 코드에서는 해당 detector 설정값을 변경하여 실험할 수 있도록 구성하였다.

(1) 모델들 설명

detector_backend	사용 목적	특징	장점	단점
retinaface	crop 이미지에서 얼굴 위치 검출	딥러닝 기반 얼굴 검출기	정면 및 일부 측면 얼굴 검출이 비교적 안정적	처리 시간이 길어질 수 있음
opencv	crop 이미지에서 얼굴 위치 검출	OpenCV 기반 전통적 얼굴 검출 방식	속도가 빠르고 가볍다	정면 얼굴 위주로 강하고, 측면·원거리에는 약함
mediapipe	crop 이미지에서 얼굴 위치 검출	실시간 영상 처리에 적합한 얼굴 검출 방식	빠르고 실시간 확장에 유리	현재 crop 이미지 조건에서는 얼굴 검출 실패가 많았음

(2) 모델 사용

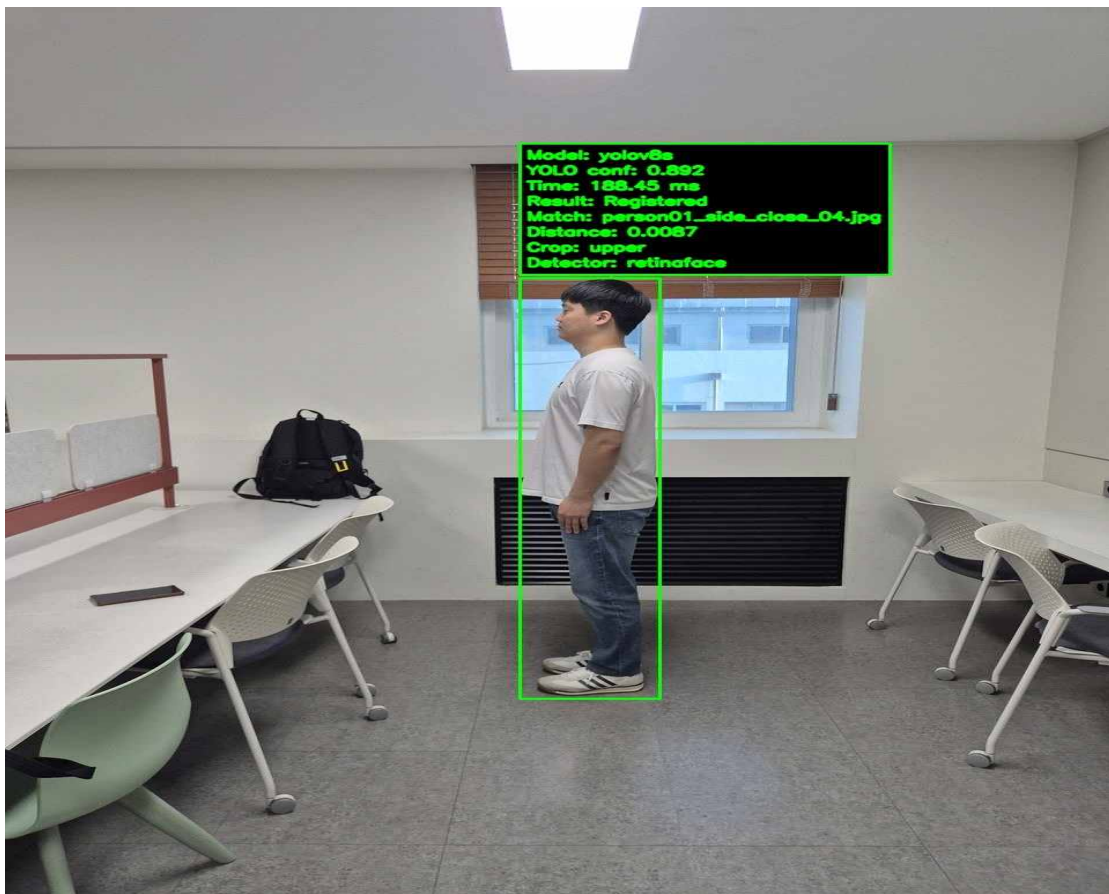
1) DETECTOR_BACKENDS = ["mediapipe"]

DeepFace의 mediapipe detector가 crop 이미지에서 얼굴을 제대로 검출하지 못합니다.

person01	YOLOv8n	0	0	10
person01	YOLOv8s	0	0	10
unknown_person	YOLOv8n	0	0	10
unknown_person	YOLOv8s	0	0	10

2) DETECTOR_BACKENDS = ["RetinaFace", "OpenCV"]

측면 이미지가 정상적으로 검출되는 것을 확인 할 수 있다.



출처: [deepface 사용 방법](#)

5. DB 저장 구조 설계

5.1 registered_person 테이블 설계

등록 인물 정보를 저장하는 테이블입니다.

컬럼명	설명
person_id	등록 인물 고유 번호
name	등록 인물 이름 또는 코드명
face_image_path	등록 얼굴 이미지 경로
registered_at	등록 날짜
status	등록 상태

```
CREATE TABLE registered_person (  
    person_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    face_image_path VARCHAR(255) NOT NULL,  
    registered_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) DEFAULT 'active'  
);
```

5.2 detection_log 테이블 설계

탐지 결과 기록을 저장하는 테이블이다.

컬럼명	설명
log_id	탐지 로그 고유 번호
detected_at	탐지 시간
test_group	테스트 그룹: person01 / unknown_person
model_name	사용 모델: yolov8n / yolov8s
image_path	입력 이미지 경로
person_detected	사람 탐지 여부
yolo_confidence	YOLO 사람 탐지 confidence
yolo_inference_ms	YOLO 탐지 시간(ms)
recognized_name	가장 유사한 등록 인물 또는 이미지
face_distance	DeepFace distance 값
is_registered	등록 인물 여부
final_result	Registered / Unknown / Unidentified

```
CREATE TABLE detection_log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    detected_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    test_group VARCHAR(50),  
    model_name VARCHAR(20),  
    image_path VARCHAR(255),  
    person_detected BOOLEAN,  
    yolo_confidence FLOAT,  
    yolo_inference_ms FLOAT,  
    recognized_name VARCHAR(100),  
    face_distance FLOAT,  
    is_registered BOOLEAN,  
    final_result VARCHAR(50)  
);
```

6. 6주차 결과 및 한계

6주차에서는 YOLOv8n과 YOLOv8s를 이용하여 입력 이미지에서 person 객체를 탐지하고, 탐지된 사람 영역을 crop하여 DeepFace ArcFace 모델과 결합하는 구조를 구현하였다.

등록 얼굴 데이터와 crop 이미지를 비교하여 Registered, Unknown, Unidentified로 분류하였으며, 결과 이미지를 별도로 저장하여 시각적으로 확인할 수 있도록 하였다.

실험 결과, 얼굴이 명확하게 보이는 정면 이미지에서는 등록 인물이 정상적으로 Registered로 판별되었고, 미등록 인물은 threshold 0.50 기준에 따라 Unknown으로 분류되었다.

YOLOv8n과 YOLOv8s 모두 person 탐지는 안정적으로 수행되었으며, YOLOv8s가 일부 이미지에서 더 높은 confidence를 보였다.

그러나 뒷모습, 원거리, 얼굴이 가려진 이미지에서는 YOLO가 사람을 탐지하더라도 DeepFace가 얼굴을 검출하지 못해 Unidentified로 분류되는 경우가 있었다.

또한 DeepFace의 detector_backend를 적용하는 과정에서 처리 시간이 증가하는 한계를 확인하였다.

RetinaFace, OpenCV, MediaPipe 등 여러 detector_backend를 비교한 결과, detector를 여러 개 순차적으로 적용할수록 얼굴 검출 가능성은 높아질 수 있으나, 등록 얼굴 이미지와 반복 비교하는 횟수가 증가하여 전체 처리 시간이 길어졌다.

특히 등록 얼굴 이미지 수가 많을수록 crop 이미지와 등록 얼굴 간 비교 횟수가 증가하여 실시간 시스템 적용에는 부담이 될 수 있음을 확인하였다.

따라서 최종 시스템에서는 모든 detector를 동시에 사용하는 방식보다, 정확도와 처리 시간을 고려하여 적절한 detector_backend를 선택해야 한다.

정확도 측면에서 RetinaFace가 비교적 안정적이었으나, 처리 시간 단축을 위해 등록 얼굴 이미지를 대표 이미지 중심으로 선별하고, 불필요한 detector 반복 적용을 줄이는 개선이 필요하다고 판단하였다.

7. 7주차 계획

7주차에서는 6주차에서 구현한 YOLO와 DeepFace 결합 구조를 개선하고, 판별 결과 저장 기능을 구체화할 계획이다.

먼저 등록 얼굴 데이터와 테스트 이미지를 정리하여 등록 데이터는 얼굴이 명확한 이미지 중심으로 구성하고, 전신·원거리·뒷모습 이미지는 테스트 이미지로 분리할 예정이다.(거리 설정도 중요)

detector_backend 적용에 따른 처리 시간 증가 문제를 개선할 계획이다. 이를 위해 등록 얼굴 이미지를 대표 이미지 중심으로 정리하고, 여러 detector를 반복 적용하는 방식보다 RetinaFace 중심의 단순화된 구조를 우선 적용할 예정이다.

또한 YOLO 탐지 시간(ms)과 DeepFace 판별 결과를 함께 비교하여, 정확도와 처리 속도를 모두 고려한 최적 조합을 검토할 계획이다.

추가적으로 이미지 기반 판별 구조를 라이브 스트리밍 환경으로 확장하는 방안도 구현할 계획이다.

현재는 정적 이미지 파일을 대상으로 YOLO 객체 탐지와 DeepFace 얼굴 인식을 수행하였으나, 최종 시스템에서는 웹캠 또는 Raspberry Pi에서 입력되는 영상 프레임을 실시간으로 처리해야 한다.

따라서 FastAPI 기반 Python 웹서버를 활용하여 YOLO 탐지 결과와 얼굴 인식 결과가 표시된 영상을 실시간으로 확인하는 구조를 구현할 예정이다.

이후 현재 CSV로 저장되는 탐지 결과를 MySQL의 detection_log 테이블과 연동하여, 탐지 시간, 모델명, YOLO confidence, 탐지 시간(ms), DeepFace distance, 최종 판별 결과를 DB에 저장하는 기능을 구현할 계획이다.

최종적으로 7주차에서는 등록/미등록 인물 판별 구조를 안정화하고, 이벤트 및 알림 기능 구현을 위한 기반을 마련하고자 한다.